

Danh sách liên kết

Sau đây là một số bài tập liên quan tới chủ đề của chương này

Chu Quang Cường

24520236

Bài 1(BT#04): So sánh các ưu khuyết điểm của

a/ Linked List có và không có con trỏ Tail luôn lưu địa chỉ Node cuối.

b/ Linked List có và không có Header Node.

Giải:

a, Ta sẽ lập bảng để so sánh hai Linked List luôn lưu địa chỉ Node cuối nhưng 1 cái có và 1 cái không có con trỏ Tail này:

Đặc điểm Con Trỏ Tail	Ưu điểm	Khuyết điểm
Có	- Vì có con trỏ Tail trỏ trực tiếp đến node cuối nên khi thêm phần tử vào cuối danh sách có độ phức tạp $O(1)$. - Hữu dụng khi cần truy cập liên tục hay thao tác nhanh ở cuối danh sách.	- Tốn thêm bộ nhớ. - Tăng độ phức tạp khi phải cập nhật con trỏ Tail ngay khi có thao tác ở cuối và đặc biệt là khi danh sách này rỗng hoặc chỉ có một phần tử.
Không có	- Tiết kiệm bộ nhớ. - Nếu không cần truy cập nhanh đến phần tử cuối thì làm cho việc sử dụng đơn giản và hiệu quả hơn.	- Vì phải duyệt từ đầu đến cuối để tìm node cuối cùng nên khi thêm phần tử vào cuối danh sách có độ phức tạp $O(n)$. - Không hiệu quả khi cần thực hiện thường xuyên các thao tác ở các phần tử cuối.

b, Tương tự, ta cũng lập bảng như sau để so sánh:

Header Node \ Đặc điểm	Ưu điểm	Khuyết điểm
Có	- Đơn giản việc thao tác phần tử ở đầu danh sách vì không cần thay đổi con trỏ head. - Xử lý danh sách rỗng dễ dàng hơn (Header Node luôn tồn tại, không cần kiểm tra NULL).	- Tốn thêm bộ nhớ. - Tăng độ phức tạp trong thiết kế khi phải phân biệt Header Node với các node dữ liệu khác.
Không có	- Tiết kiệm bộ nhớ. - Dễ hiểu và vận hành.	- Xử lý trường hợp danh sách rỗng phức tạp hơn (phải kiểm tra con trỏ ở đầu danh sách phải là NULL). - Cần thao tác và quản lý cẩn thận vì khi thêm phần tử vào đầu danh sách cần thay đổi con trỏ Head.

Bài 2(BT#05): Viết hàm chuyển danh sách sinh viên đang lưu trong một mảng động vào

- a) danh sách liên kết đơn không có con trỏ cuối và ngược lại.
- b) danh sách liên kết đơn luôn có con trỏ cuối và ngược lại.
- c) danh sách liên kết kép và ngược lại.
- d) danh sách liên kết vòng và ngược lại.
- e) danh sách liên kết kép vòng và ngược lại.

Giải:

Đầu tiên, ta lập cấu trúc cho danh sách liên kết là Student với Node, còn mảng động là Student arr[] với thuộc tính MSSV và name.

```
struct Student
{
    int MSSV;
    string name;
};

struct Node
{
    Student data;
    Node *next;
    Node *pre;
};

struct LinkedList
{
    Node *head;
    Node *tail;
};
```

a, Sau đây là hàm chuyển danh sách sinh viên đang lưu trong một mảng động vào danh sách liên kết đơn không có con trỏ cuối và ngược lại:

```
Node *ArrayToSLL(Student *arr, int n)
{
    if (n == 0)
        return NULL;

    Node *head = new Node{arr[0], NULL};
    Node *curr = head;

    for (int i = 1; i < n; i++)
    {
        curr->next = new Node{arr[i], NULL};
        curr = curr->next;
    }
    return head;
}

Student *SLLToArray(Node *head, int &n)
{
    n = 0;
    Node *temp = head;
    while (temp)
    {
        n++;
        temp = temp->next;
    }

    Student *arr = new Student[n];
    temp = head;
    for (int i = 0; i < n; i++)
    {
        arr[i] = temp->data;
        temp = temp->next;
    }
    return arr;
}
```

b, Vào danh sách liên kết đơn luôn có con trỏ cuối:

```
LinkedList ArrayToSLLWithTail(Student *arr, int n)
{
    LinkedList list = {NULL, NULL};

    if (n == 0)
        return list;

    list.head = list.tail = new Node{arr[0], NULL};

    for (int i = 1; i < n; i++)
    {
        list.tail->next = new Node{arr[i], NULL};
        list.tail = list.tail->next;
    }
    return list;
}

Student *SLLWithTailToArray(LinkedList *list, int &n)
{
    n = 0;

    Node *head = list->head;
    Node *temp = head;
    while (temp)
    {
        n++;
        head = temp->next;
    }

    Student *arr = new Student[n];
    temp = head;
    for (int i = 0; i < n; i++)
    {
        arr[i] = temp->data;
        temp = temp->next;
    }
    return arr;
}
```

c, Sau đây là hàm chuyển danh sách sinh viên đang lưu trong một mảng động vào danh sách liên kết kép:

```
LinkedList ArrayToDLL(Student* arr, int n)
{
    LinkedList list = {NULL, NULL};

    if (n == 0)
        return list;

    list.head = list.tail = new Node{arr[0], NULL, NULL};

    for (int i = 1; i < n; i++)
    {
        list.tail->next = new Node{arr[i], NULL, list.tail};
        list.tail = list.tail->next;
    }
    return list;
}
```

d, Sau đây là hàm chuyển danh sách sinh viên đang lưu trong một mảng động vào danh sách liên kết vòng:

```
LinkedList ArrayToCLL(Student *arr, int n)
{
    LinkedList list = {NULL, NULL};

    if (n == 0)
        return list;

    list.head = list.tail = new Node{arr[0], NULL};

    for (int i = 1; i < n; i++)
    {
        list.tail->next = new Node{arr[i], NULL};
        list.tail = list.tail->next;
    }
    list.tail->next = list.head;
    return list;
}
```

d, Sau đây là hàm chuyển danh sách sinh viên đang lưu trong một mảng động vào danh sách liên kết kép vòng:

```
LinkedList ArrayToCDLL(Student *arr, int n)
{
    LinkedList list = {NULL, NULL};

    if (n == 0)
        return list;

    list.head = list.tail = new Node{arr[0], NULL, NULL};

    for (int i = 1; i < n; i++)
    {
        list.tail->next = new Node{arr[i], NULL, NULL};
        list.tail = list.tail->next;
    }
    list.tail->next = list.head;
    list.head->pre = list.tail;
    return list;
}
```

Phần ngược lại của 3 câu c, d, e thì ta làm tương tự như câu b.

Bài 3(BT#06): Viết hàm thêm 1 phần tử vào

a) mảng động đang có thứ tự.

b) xâu đơn có thứ tự + có và không có con trỏ cuối (2 hàm khác nhau).

c) xâu đơn có thứ tự + có Header node + có và không có con trỏ cuối (2 hàm khác nhau).

d) xâu đơn có thứ tự + có Header node và luôn có địa chỉ node cuối nằm trong các byte đầu của trường dữ liệu trong Header node.

Giải:

Ở đây, ta cũng thiết lập cấu trúc Node và danh sách đơn:

```
struct Node
{
    char data;
    Node *next;
    Node *pre;
};

struct LinkedList
{
    Node *head;
    Node *tail;
};
```

a, Hàm thêm 1 phần tử vào mảng động đang có thứ tự là:

```
void MangDongCoThuTu(int *&arr, int &n, char c, int pos)
{
    if (pos < 0 || pos > n)
        return;
    int *newArr = new int[n + 1];
    for (int i = 0; i < pos; i++)
        newArr[i] = arr[i];
    newArr[pos] = c;
    for (int i = pos + 1; i <= n; i++)
        newArr[i] = arr[i - 1];
    delete[] arr;
    arr = newArr;
    n++;
}
```

b, Hàm thêm 1 phần tử vào xâu đơn có thứ tự + không có con trỏ cuối:

```
void XauCoTTKhongCTCuoi(Node *&head, char c, int pos){
    Node *newNode = new Node{c, NULL};
    if (pos == 0) {
        newNode->next = head;
        head = newNode;
        return;
    }
    Node *curr = head;
    for (int i = 0; curr && i < pos - 1; i++)
        curr = curr->next;
    if (curr) {
        newNode->next = curr->next;
        curr->next = newNode;
    }
}
```

Hàm thêm 1 phần tử vào xâu đơn có thứ tự + có con trỏ cuối:

```
void XauCoTTCuoi(LinkedList &list, char c, int pos)
{
    Node *newNode = new Node{c, NULL};
    if (pos == 0)
    {
        newNode->next = list.head;
        list.head = newNode;
        if (!list.tail)
            list.tail = newNode;
        return;
    }
    Node *curr = list.head;
    for (int i = 0; curr && i < pos - 1; i++)
        curr = curr->next;
    if (curr)
    {
        newNode->next = curr->next;
        curr->next = newNode;
        if (!newNode->next)
            list.tail = newNode;
    }
}
```

c, Hàm thêm phần tử vào xâu đơn có thứ tự và Header node, không có con trỏ cuối:

```
void Xau_c1(LinkedList &list, char c, int pos)
{
    Node *newNode = new Node{c, NULL};
    Node *curr = list.head;
    for (int i = 0; curr && i < pos - 1; i++)
        curr = curr->next;
    if (pos == 0)
    {
        newNode->next = list.head;
        list.head = newNode;
    }
    else if (curr)
    {
        newNode->next = curr->next;
        curr->next = newNode;
    }
}
```

Hàm thêm phần tử vào xâu đơn có thứ tự và Header node, có con trỏ cuối:

```
void Xau_c2(LinkedList &list, char c, int pos)
{
    Node *newNode = new Node{c, NULL};
    if (pos == 0)
    {
        newNode->next = list.head;
        list.head = newNode;
        if (!list.tail)
            list.tail = newNode;
        return;
    }
    Node *curr = list.head;
    for (int i = 0; curr && i < pos - 1; i++)
        curr = curr->next;
    if (curr)
    {
        newNode->next = curr->next;
        curr->next = newNode;
        if (!newNode->next)
            list.tail = newNode;
    }
}
```

d, Hàm thêm 1 phần tử vào xâu đơn có thứ tự + có Header node và luôn có địa chỉ node cuối nằm trong các byte đầu của trường dữ liệu trong Header node.

```
struct LinkedListSpecial
{
    Node *head;
    Node **tail;
};

void Xau_d(LinkedListSpecial &list, char c, int pos)
{
    Node *newNode = new Node{c, NULL};

    if (pos == 0)
    {
        newNode->next = list.head;
        list.head = newNode;
        if (!*list.tail)
            *list.tail = newNode;
        return;
    }

    Node *curr = list.head;
    for (int i = 0; curr && i < pos - 1; i++)
        curr = curr->next;

    if (curr)
    {
        newNode->next = curr->next;
        curr->next = newNode;
        if (!newNode->next)
            *list.tail = newNode;
    }
}
```